

LAPORAN PRAKTIKUM MONGODB

Laporan Hari ke-1



Disusun oleh:

Nama: Muhammad Dhafin Rizqullah

NIM: 225443015

Kelas: 1AEC-1

Program Studi: Teknologi Rekayasa Informatika Industri

Jurusan: Teknik Otomasi Manufaktur dan Mekatronika

1AEC-1

TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI

POLITEKNIK MANUFAKTUR BANDUNG

TAHUN AJARAN 2025/2026

Pendahuluan

Praktikum ini bertujuan untuk memahami dasar operasi MongoDB khususnya dalam pengolahan data menggunakan aggregation pipeline. Mahasiswa diharapkan mampu melakukan pembuatan data dummy, pengolahan data produksi dan inspeksi, serta menganalisis performa mesin melalui perhitungan metrik seperti OEE (Overall Equipment Effectiveness). Selain itu, praktikum ini juga melatih kemampuan dalam mengolah data skala besar untuk menghasilkan informasi yang berguna dalam pengambilan keputusan di lingkungan industri.

Desain Data

Diagram Koleksi inspeksi

Nama Field	Tipe Data	Contoh Nilai	Keterangan
_id	ObjectId	(auto)	Primary key otomatis MongoDB
batch_id	String	B0001 – B2000	Kode batch produksi (foreign key ke target_kualitas)
mesin	String	M01 – M10	Mesin yang menghasilkan batch tersebut
tanggal	Date	2026-05-03T14:30Z	Tanggal dan jam inspeksi dilakukan
shift	Integer	1 / 2 / 3	Shift saat inspeksi (diturunkan dari jam)
inspektor	String	Andi, Budi, dst.	Nama inspektor yang bertugas
jumlah_diperiksa	Integer	350	Total unit yang diperiksa dalam batch
cacat_ditemukan	Integer	14	Jumlah unit cacat yang ditemukan
jenis_cacat	Array<String>	["gores", "retak"]	Tipe cacat yang ditemukan (multi-value)

Diagram Koleksi target_kualitas

Nama Field	Tipe Data	Contoh Nilai	Keterangan
_id	ObjectId	(auto)	Primary key otomatis MongoDB
batch_id	String	B0001 – B2000	Kode batch (primary key logis, index unik)
target_maks_cacat	Integer	1 – 15 (%)	Batas maksimal persentase cacat yang diizinkan

Diagram Koleksi data_sensor

Nama Field	Tipe Data	Contoh Nilai	Keterangan
_id	ObjectId	(auto)	Primary key otomatis MongoDB
sensor_id	String	S01 – S20	ID sensor (20 sensor, padStart 2 digit)
nilai	Integer	0 – 100	Nilai pembacaan sensor (skala 0–100)
timestamp	Date	2026-06-28T14:22Z	Waktu pembacaan sensor (7 hari terakhir)
category	String	suhu/tekanan/getaran	Kategori sensor: suhu, tekanan, atau getaran

Diagram Koleksi sensor_info

Nama Field	Tipe Data	Contoh Nilai	Keterangan
_id	ObjectId	(auto)	Primary key otomatis MongoDB
sensor_id	String	S01 – S05	ID sensor (foreign key ke data_sensor)
nama	String	Sensor Suhu A	Nama deskriptif sensor (hanya 5 terdaftar)

Pipeline dan Hasil

1. Pipeline 1: Persentase Cacat per Batch

```
db.inspeksi.aggregate([
  {
    $addFields: {
      persen_cacat: {
        $multiply: [
          { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] },
          100
        ]
      }
    }
  },
  { $sort: { persen_cacat: -1 } },
  { $limit: 10 }
])
```

>_MONGOSH

])

< {

```
  _id: ObjectId('69f8370293634fca2b1b8cb5'),
  batch_id: 'B0556',
  mesin: 'M04',
  tanggal: 2026-05-28T11:48:00.000Z,
  shift: 2,
  inspektor: 'Budi',
  jumlah_diperiksa: 403,
  cacat_ditemukan: 40,
  jenis_cacat: [
    'dimensi'
  ],
  persen_cacat: 9.925558312655088
}
```

{

```
  _id: ObjectId('69f8370293634fca2b1b8db8'),
  batch_id: 'B2000',
  mesin: 'M02',
  tanggal: 2026-06-20T15:30:00.000Z,
  shift: 3,
  inspektor: 'Andi',
  jumlah_diperiksa: 363,
  cacat_ditemukan: 36,
  jenis_cacat: [
    'warna',
    'bengkok'
  ],
  persen_cacat: 9.917355371900827
}
```

{

```
  _id: ObjectId('69f8370293634fca2b1b8ae4'),
  batch_id: 'B1661',
  mesin: 'M07',
  tanggal: 2026-06-06T19:22:00.000Z,
  shift: 1,
  inspektor: 'Citra',
  jumlah_diperiksa: 212,
  cacat_ditemukan: 21,
  jenis_cacat: [
    'dimensi',
    'bengkok'
  ]
}
```

>_MONGOSH

```
    persen_cacat: 9.90566037735849
  }
  {
    _id: ObjectId('69f8370293634fca2b1b8cd3'),
    batch_id: 'B0119',
    mesin: 'M07',
    tanggal: 2026-05-16T10:03:00.000Z,
    shift: 2,
    inspektor: 'Eka',
    jumlah_diperiksa: 394,
    cacat_ditemukan: 39,
    jenis_cacat: [
      'retak'
    ],
    persen_cacat: 9.898477157360407
  }
  {
    _id: ObjectId('69f8370293634fca2b1b8b8e'),
    batch_id: 'B1377',
    mesin: 'M06',
    tanggal: 2026-05-02T01:25:00.000Z,
    shift: 1,
    inspektor: 'Citra',
    jumlah_diperiksa: 293,
    cacat_ditemukan: 29,
    jenis_cacat: [
      'bengkok',
      'pori'
    ],
    persen_cacat: 9.897610921501707
  }
}
```

```
{
  _id: ObjectId('69f8370293634fca2b1b8f81'),
  batch_id: 'B0155',
  mesin: 'M01',
  tanggal: 2026-05-10T13:20:00.000Z,
  shift: 2,
  inspektor: 'Eka',
  jumlah_diperiksa: 273,
  cacat_ditemukan: 27,
  jenis_cacat: [
    'gores',
    'retak',
    'bengkok'
  ],
  persen_cacat: 9.89010989010989
}
{
  _id: ObjectId('69f8370293634fca2b1b8b97'),
  batch_id: 'B1446',
  mesin: 'M03',
  tanggal: 2026-04-13T21:05:00.000Z,
  shift: 1,
  inspektor: 'Citra',
  jumlah_diperiksa: 385,
  cacat_ditemukan: 38,
  jenis_cacat: [
    'retak',
    'warna',
    'bengkok'
  ],
  persen_cacat: 9.87012987012987
}
```

```
{
  _id: ObjectId('69f8370293634fca2b1b8b33'),
  batch_id: 'B1461',
  mesin: 'M10',
  tanggal: 2026-04-05T18:44:00.000Z,
  shift: 1,
  inspektor: 'Citra',
  jumlah_diperiksa: 264,
  cacat_ditemukan: 26,
  jenis_cacat: [
    'dimensi',
    'bengkok',
    'retak'
  ],
  persen_cacat: 9.848484848484848
}
{
  _id: ObjectId('69f8370293634fca2b1b8fbb'),
  batch_id: 'B0274',
  mesin: 'M10',
  tanggal: 2026-05-13T17:27:00.000Z,
  shift: 1,
  inspektor: 'Budi',
  jumlah_diperiksa: 458,
  cacat_ditemukan: 45,
  jenis_cacat: [
    'bengkok',
    'warna',
    'pori'
  ],
  persen_cacat: 9.82532751091703
}
```



```
{
  _id: ObjectId('69f8370293634fca2b1b8f0b'),
  batch_id: 'B0770',
  mesin: 'M06',
  tanggal: 2026-05-17T13:17:00.000Z,
  shift: 2,
  inspektor: 'Andi',
  jumlah_diperiksa: 285,
  cacat_ditemukan: 28,
  jenis_cacat: [
    'pori',
    'warna'
  ],
  persen_cacat: 9.824561403508772
}
```

2. Pipeline 2: Gabung dengan Target dan Tentukan Status Batch

```
db.inspeksi.aggregate([
  {
    $lookup: {
      from: "target_kualitas",
      localField: "batch_id",
      foreignField: "batch_id",
      as: "target_info"
    }
  },
  {
    $unwind: {
      path: "$target_info",
      preserveNullAndEmptyArrays: false
    }
  },
  {
    $addFields: {
      persen_cacat: {
```

```

$cond: [
  { $eq: ["$jumlah_diperiksa", 0] },
  0,
  {
    $multiply: [
      { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] },
      100
    ]
  }
],
target_maks: "$target_info.target_maks_cacat"
}
},
{
  $addFields: {
    status: {
      $cond: {
        if: { $gt: ["$persen_cacat", "$target_maks"] },
        then: "NOT OK",
        else: "OK"
      }
    }
  }
},
{
  $project: {
    _id: 0,
    batch_id: 1,
    mesin: 1,
    persen_cacat: 1,
    target_maks: 1,
    status: 1,
    selisih: {
      $subtract: ["$persen_cacat", "$target_maks"]
    }
  }
}

```

```
},  
{ $sort: { selisih: -1 } },  
{ $limit: 10 }
```

)

>_MONGOSH

```
< {
  batch_id: 'B0152',
  mesin: 'M04',
  persen_cacat: 9.641255605381167,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.641255605381167
}
{
  batch_id: 'B0417',
  mesin: 'M10',
  persen_cacat: 9.5,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.5
}
{
  batch_id: 'B1123',
  mesin: 'M05',
  persen_cacat: 9.49074074074074,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.49074074074074
}
{
  batch_id: 'B0821',
  mesin: 'M07',
  persen_cacat: 9.420289855072465,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.420289855072465
}
{
  batch_id: 'B0890',
  mesin: 'M08',
  persen_cacat: 9.408602150537634,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.408602150537634
}
```

```
{
  batch_id: 'B1039',
  mesin: 'M07',
  persen_cacat: 9.251101321585903,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.251101321585903
}
{
  batch_id: 'B1295',
  mesin: 'M02',
  persen_cacat: 9.210526315789473,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.210526315789473
}
{
  batch_id: 'B1441',
  mesin: 'M01',
  persen_cacat: 9.130434782608695,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.130434782608695
}
{
  batch_id: 'B1925',
  mesin: 'M03',
  persen_cacat: 9.01639344262295,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.01639344262295
}
{
  batch_id: 'B0305',
  mesin: 'M01',
  persen_cacat: 8.921161825726141,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 7.921161825726141
}
```

3. Pipeline 3: Jumlah Batch NOT OK per Mesin per Minggu

```
db.inspeksi.aggregate([
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100 ] }
  }},
  { $addFields: {
    status: { $cond: { if: { $gt: ["$persen_cacat", "$target_info.target_maks_cacat"] }, then: "NOT
OK", else: "OK" } }
  }},
  { $match: { status: "NOT OK" } },
  { $group: {
    _id: {
      mesin: "$mesin",
      minggu: { $week: "$tanggal" },
      tahun: { $year: "$tanggal" }
    },
    jumlah_NOT_OK: { $sum: 1 }
  }},
  { $sort: { "_id.tahun": 1, "_id.minggu": 1, "_id.mesin": 1 } }
])
```

>_MONGOSH

```
< {
  _id: {
    mesin: 'M01',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 3
}
{
  _id: {
    mesin: 'M02',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 3
}
{
  _id: {
    mesin: 'M04',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 1
}
{
  _id: {
    mesin: 'M05',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 3
}
{
  _id: {
    mesin: 'M06',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 5
}
```

```
{
  _id: {
    mesin: 'M07',
    minggu: 14,
    tahun: 2026
  },
  jumlah_NOT_OK: 2
}
{
  _id: {
    mesin: 'M08',
    minggu: 14,
    tahun: 2026
  },
  jumlah_NOT_OK: 5
}
{
  _id: {
    mesin: 'M09',
    minggu: 14,
    tahun: 2026
  },
  jumlah_NOT_OK: 5
}
{
  _id: {
    mesin: 'M10',
    minggu: 14,
    tahun: 2026
  },
  jumlah_NOT_OK: 1
}
```


4. Pipeline 4: Top 3 Mesin dengan Jumlah NOT OK Terbanyak (dalam satu bulan tertentu)

```
db.inspeksi.aggregate([
  { $match: {
    tanggal: { $gte: ISODate("2026-05-01"), $lt: ISODate("2026-06-01") }
  } },
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  } },
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100 ] }
  } },
  { $addFields: {
    status: { $cond: { if: { $gt: ["$persen_cacat", "$target_info.target_maks_cacat"] }, then: "NOT
OK", else: "OK" } }
  } },
  { $match: { status: "NOT OK" } },
  { $group: {
    _id: "$mesin",
    total_NOT_OK: { $sum: 1 }
  } },
  { $sort: { total_NOT_OK: -1 } },
  { $limit: 3 }
])
```

```
{
  _id: 'M01',
  total_NOT_OK: 23
}
{
  _id: 'M09',
  total_NOT_OK: 19
}
{
  _id: 'M05',
  total_NOT_OK: 19
}
{
  _id: 'M02',
  total_NOT_OK: 17
}
{
  _id: 'M04',
  total_NOT_OK: 16
}
{
  _id: 'M07',
  total_NOT_OK: 15
}
{
  _id: 'M10',
  total_NOT_OK: 12
}
{
  _id: 'M08',
  total_NOT_OK: 12
}
{
  _id: 'M03',
  total_NOT_OK: 11
}
{
  _id: 'M06',
  total_NOT_OK: 7
}
```

5. Pipeline 5 (Bonus/Opsional): Ringkasan Bulanan dengan \$facet

```
db.inspeksi.aggregate([
  { $match: { tanggal: { $gte: ISODate("2026-06-01"), $lt: ISODate("2026-07-01") } } },
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100 ] },
    status: { $cond: { if: { $gt: [ { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100 ] }, "$target_info.target_maks_cacat" ] }, then: "NOT OK", else: "OK" } } }
  }},
  { $facet: {
    "total_batch": [
      { $count: "count" }
    ],
    "rata_cacat": [
      { $group: { _id: null, avg_cacat: { $avg: "$persen_cacat" } } }
    ],
    "mesin_terburuk": [
      { $match: { status: "NOT OK" } },
      { $group: { _id: "$mesin", jumlah: { $sum: 1 } } },
      { $sort: { jumlah: -1 } },
      { $limit: 3 }
    ]
  }
}]
```

D)

```
{
  total_batch: [
    {
      count: 478
    }
  ],
  rata_cacat: [
    {
      _id: null,
      avg_cacat: 4.697990449982752
    }
  ],
  mesin_terburuk: [
    {
      _id: 'M05',
      jumlah: 21
    },
    {
      _id: 'M10',
      jumlah: 16
    },
    {
      _id: 'M07',
      jumlah: 15
    },
    {
      _id: 'M01',
      jumlah: 14
    },
    {
      _id: 'M06',
      jumlah: 13
    },
    {
      _id: 'M02',
      jumlah: 12
    },
    {
      _id: 'M03',
      jumlah: 12
    },
    {
      _id: 'M04',
      jumlah: 12
    },
    {
      _id: 'M08',
      jumlah: 11
    },
    {
      _id: 'M09',
      jumlah: 11
    }
  ]
}
```

Analisis dan Kesimpulan

Analisis

Berdasarkan hasil pengolahan data produksi harian, terlihat bahwa nilai OEE tiap mesin bervariasi pada setiap bulan. Mesin dengan nilai OEE rendah (di bawah 80%) menunjukkan adanya masalah pada salah satu atau lebih komponen utama, yaitu availability, performance, atau quality.

Dari pipeline yang dijalankan, mesin dengan OEE terendah cenderung memiliki rasio actual_ok terhadap target yang kecil, serta jumlah reject yang cukup tinggi. Hal ini menunjukkan bahwa faktor kualitas dan performa menjadi penyebab utama rendahnya efektivitas mesin. Selain itu, dari distribusi produksi per shift, terlihat bahwa beberapa shift memiliki output yang lebih rendah, yang dapat disebabkan oleh perbedaan operator, kondisi mesin, atau waktu operasional.

Pada analisis data inspeksi, ditemukan bahwa beberapa batch memiliki persentase cacat yang melebihi target kualitas yang telah ditentukan. Mesin yang sering menghasilkan status NOT OK secara konsisten dalam periode tertentu (misalnya mingguan atau bulanan) dapat diidentifikasi sebagai mesin yang bermasalah. Pipeline juga menunjukkan adanya mesin dengan jumlah batch NOT OK tertinggi, yang mengindikasikan perlunya evaluasi proses produksi, perawatan mesin, atau peningkatan kontrol kualitas.

Selain itu, dari analisis data sensor, beberapa sensor menunjukkan nilai ekstrem (di atas 90) lebih dari 3 kali, yang dapat menjadi indikasi adanya anomali atau kondisi operasional yang tidak stabil. Rata-rata nilai per jam juga menunjukkan pola tertentu yang bisa digunakan untuk mengidentifikasi waktu-waktu kritis dalam proses produksi.

Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa penggunaan aggregation pada MongoDB sangat efektif dalam menganalisis data produksi dan kualitas secara cepat dan terstruktur. Hasil analisis menunjukkan bahwa terdapat beberapa mesin yang memiliki nilai OEE rendah dan sering menghasilkan produk dengan kualitas di bawah standar, sehingga mesin-mesin tersebut perlu menjadi prioritas utama untuk dilakukan perbaikan atau maintenance.

Selain itu, faktor shift kerja dan kondisi operasional juga berpengaruh terhadap hasil produksi. Data inspeksi menunjukkan bahwa pengendalian kualitas masih perlu ditingkatkan, terutama pada batch dengan persentase cacat tinggi. Dengan memanfaatkan data sensor, potensi masalah juga dapat dideteksi lebih awal melalui pola nilai yang tidak normal.

Secara keseluruhan, analisis ini membantu dalam pengambilan keputusan berbasis data untuk meningkatkan efisiensi produksi, kualitas produk, serta kinerja mesin di lingkungan industri.

Lampiran

<https://drive.google.com/file/d/1H7kCpgHm0NFgCUUyMkLPpHaZEDQveg8/view?usp=sharing>

TUTORIAL TERBIMBING STUDI KASUS AGREGASI

1. Buat Koleksi dan Generate Data Dummy

```
db.produksi_harian.drop();
```

```
var docs = [];
```

```
var mesinList = [];
```

```
for (let i = 1; i <= 10; i++) {  
  mesinList.push("M" + i.toString().padStart(2, '0'));  
}
```

```
var start = new Date(2026, 3, 1);
```

```
for (let i = 0; i < 1000; i++) {  
  var tgl = new Date(  
    start.getTime() + Math.floor(Math.random() * 90) * 86400000  
  );
```

```
  var shift = Math.floor(Math.random() * 3) + 1;
```

```
  var target = Math.floor(Math.random() * 301) + 200;
```

```
  var actual_ok = Math.floor(target * (0.6 + Math.random() * 0.4));
```

```
  var actual_reject = Math.floor(actual_ok * Math.random() * 0.15);
```

```
  var durasi_tersedia = 480;
```

```
  var durasi_operasi = Math.floor(  
    durasi_tersedia * (0.7 + Math.random() * 0.3)  
  );
```

```
  docs.push({
```

```

        mesin: mesinList[Math.floor(Math.random() * 10)],
        tanggal: tgl,
        shift: shift,
        target: target,
        actual_ok: actual_ok,
        actual_reject: actual_reject,
        durasi_operasi_menit: durasi_operasi,
        durasi_tersedia_menit: durasi_tersedia
    });

    if (docs.length === 500) {
        db.produksi_harian.insertMany(docs);
        docs = [];
    }
    if (docs.length) {
        db.produksi_harian.insertMany(docs);
    }
}

```

```

< []
> db.produksi_harian.countDocuments()
< 1000
> db.produksi_harian.findOne()
< {
  _id: ObjectId('69f81574d5146d7c67940c0e'),
  mesin: 'M10',
  tanggal: 2026-06-23T17:00:00.000Z,
  shift: 1,
  target: 488,
  actual_ok: 307,
  actual_reject: 22,
  durasi_operasi_menit: 413,
  durasi_tersedia_menit: 480
}

```

2. Pipeline Lengkap

```

db.produksi_harian.aggregate([
    // Tahap 1: Tidak perlu $match karena ingin semua data

```

// Tahap 2: Group by mesin dan bulan

```
{ $group: {  
  _id: {  
    mesin: "$mesin",  
    bulan: { $month: "$tanggal" },  
    tahun: { $year: "$tanggal" }  
  },  
  total_target: { $sum: "$target" },  
  total_ok: { $sum: "$actual_ok" },  
  total_reject: { $sum: "$actual_reject" },  
  total_op: { $sum: "$durasi_operasi_menit" },  
  total_avail: { $sum: "$durasi_tersedia_menit" }  
}},
```

// Tahap 3: Hitung OEE

```
{ $project: {  
  _id: 0,  
  mesin: "$_id.mesin",  
  bulan: "$_id.bulan",  
  tahun: "$_id.tahun",  
  total_ok: 1,  
  total_target: 1,  
  total_reject: 1,  
  availability: { $divide: ["$total_op", "$total_avail"] },  
  performance: { $divide: ["$total_ok", "$total_target"] },  
  quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },  
  OEE: {  
    $multiply: [  
      { $divide: ["$total_ok", "$total_target"] },  
      { $divide: ["$total_op", "$total_avail"] },  
      { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }  
    ]  
  }  
}},
```

// Tahap 4: Urutkan berdasarkan OEE terendah

```
{ $sort: { OEE: 1 } }
```

)

_MONGOSH

```
< {
  total_target: 590,
  total_ok: 405,
  total_reject: 19,
  mesin: 'M05',
  bulan: 3,
  tahun: 2026,
  availability: 0.83125,
  performance: 0.6864406779661016,
  quality: 0.9551886792452831,
  OEE: 0.5450343030460505
}
{
  total_target: 320,
  total_ok: 257,
  total_reject: 4,
  mesin: 'M04',
  bulan: 3,
  tahun: 2026,
  availability: 0.7354166666666667,
  performance: 0.803125,
  quality: 0.9846743295019157,
  OEE: 0.581579686502235
}
{
  total_target: 10303,
  total_ok: 7957,
  total_reject: 649,
  mesin: 'M02',
  bulan: 5,
  tahun: 2026,
  availability: 0.8395833333333333,
  performance: 0.7722993302921479,
  quality: 0.92458749709505,
  OEE: 0.5995114517408523
}
```

3. MENEMUKAN MESIN DENGAN OEE DI BAWAH 80%

```
db.produksi_harian.aggregate([
// Tahap 1: Tidak perlu $match karena ingin semua data
// Tahap 2: Group by mesin dan bulan
{ $group: {
  _id: {
    mesin: "$mesin",
    bulan: { $month: "$tanggal" },
    tahun: { $year: "$tanggal" }
  },
  total_target: { $sum: "$target" },
  total_ok: { $sum: "$actual_ok" },
  total_reject: { $sum: "$actual_reject" },
  total_op: { $sum: "$durasi_operasi_menit" },
  total_avail: { $sum: "$durasi_tersedia_menit" }
}},
// Tahap 3: Hitung OEE
{ $project: {
  _id: 0,
  mesin: "$_id.mesin",
  bulan: "$_id.bulan",
  tahun: "$_id.tahun",
  total_ok: 1,
  total_target: 1,
  total_reject: 1,
  availability: { $divide: ["$total_op", "$total_avail"] },
  performance: { $divide: ["$total_ok", "$total_target"] },
  quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
  OEE: {
    $multiply: [
      { $divide: ["$total_ok", "$total_target"] },
      { $divide: ["$total_op", "$total_avail"] },
      { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
    ]
  }
}},
// Tahap 4: Urutkan berdasarkan OEE terendah
{ $sort: { OEE: 1 } }
])
```

>_MONGOSH

```
< {  
  mesin: 'M05',  
  bulan: 3,  
  tahun: 2026,  
  OEE: 0.5450343030460505  
}  
{  
  mesin: 'M04',  
  bulan: 3,  
  tahun: 2026,  
  OEE: 0.581579686502235  
}  
{  
  mesin: 'M02',  
  bulan: 5,  
  tahun: 2026,  
  OEE: 0.5995114517408523  
}  
{  
  mesin: 'M03',  
  bulan: 4,  
  tahun: 2026,  
  OEE: 0.6091290295643336  
}  
{  
  mesin: 'M02',  
  bulan: 6,  
  tahun: 2026,  
  OEE: 0.6116488877540777  
}  
{  
  mesin: 'M03',  
  bulan: 6,  
  tahun: 2026,  
  OEE: 0.6118778333673225
```

4. DISTRIBUSI JUMLAH PRODUKSI PER SHIFT DALAM BUCKET

Manajemen ingin melihat sebaran jumlah produksi (actual_ok) per shift, misal dikelompokkan menjadi bucket: rendah (0-200), sedang (200-400), tinggi (400+). Gunakan \$bucket.

```
db.produksi_harian.aggregate([
  { $bucket: {
    groupBy: "$actual_ok",
    boundaries: [0, 200, 300, 400, 600], // 0-199, 200-299, 300-399, 400-600
    default: "di atas 600",
    output: {
      count: { $sum: 1 },
      rata_reject: { $avg: "$actual_reject" }
    }
  }
}]
])
```

```
< {
  _id: 0,
  count: 180,
  rata_reject: 13.15
}
{
  _id: 200,
  count: 414,
  rata_reject: 18.669082125603865
}
{
  _id: 300,
  count: 291,
  rata_reject: 24.701030927835053
}
{
  _id: 400,
  count: 115,
  rata_reject: 34.04347826086956
}
```

Namun ini belum dipisah per shift. Untuk distribusi per shift, kita bisa \$group atau \$facet. Misal kita ingin tiga bucket terpisah untuk setiap shift:

```
db.produksi_harian.aggregate([
  { $group: {
    _id: "$shift",
    data: { $push: "$actual_ok" } // kumpulkan nilai
  }},
  // Kemudian di aplikasi atau gunakan operator $bucket di sub-pipeline (tidak bisa langsung karena
  // $bucket butuh array dokumen)
])
```

```
{
  _id: 2,
  data: [
    300,
    219,
    391,
    433,
    481,
    196,
    222,
    142,
    195,
    295,
    219,
    260,
    274,
    240,
    180,
    458,
    374,
    265,
    248,
    241,
    163,
```

Gunakan \$facet untuk menjalankan pipeline bucket paralel per shift:

```
db.produksi_harian.aggregate([
  { $facet: {
    "shift1": [
      { $match: { shift: 1 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default: "600+", output: {
count: {$sum:1} } } } }
    ],
    "shift2": [
      { $match: { shift: 2 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default: "600+", output: {
count: {$sum:1} } } } }
    ],
    "shift3": [
      { $match: { shift: 3 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default: "600+", output: {
count: {$sum:1} } } } }
    ]
  } }
])
```

>_MONGOSH

```
< {
  shift1: [
    {
      _id: 0,
      count: 55
    },
    {
      _id: 200,
      count: 145
    },
    {
      _id: 300,
      count: 94
    },
    {
      _id: 400,
      count: 26
    }
  ],
  shift2: [
    {
      _id: 0,
      count: 58
    },
    {
      _id: 200,
      count: 126
    },
    {
      _id: 300,
      count: 92
    },
    {
```

Cara lain – lebih sederhana: Jika hanya ingin jumlah per shift, gunakan \$group langsung:

```
db.produksi_harian.aggregate([
```

```
  { $group: { _id: "$shift", total_produksi: { $sum: "$actual_ok" } } }
```

```
])
```

```
< {
  _id: 3,
  total_produksi: 101771
}
{
  _id: 1,
  total_produksi: 89005
}
{
  _id: 2,
  total_produksi: 92386
}
```


Latihan Tambahan

1. Tren Bulanan: Modifikasi pipeline agar menampilkan OEE per bulan

untuk semua mesin, lalu urutkan bulan.

```
db.produksi_harian.aggregate([
  { $group: {
    _id: "$shift",
    data: { $push: "$actual_ok" } // kumpulkan nilai
  }},
  // Kemudian di aplikasi atau gunakan operator $bucket di sub-pipeline (tidak
  // bisa langsung karena $bucket butuh array dokumen)
])
db.produksi_harian.aggregate([
  {
    $group: {
      _id: {
        mesin: "$mesin",
        bulan: { $month: "$tanggal" },
        tahun: { $year: "$tanggal" }
      },
      total_target: { $sum: "$target" },
      total_ok: { $sum: "$actual_ok" },
      total_reject: { $sum: "$actual_reject" },
      total_op: { $sum: "$durasi_operasi menit" },
      total_avail: { $sum: "$durasi_tersedia menit" }
    }
  },
  {
    $project: {
      _id: 0,
      mesin: "$_id.mesin",
      bulan: "$_id.bulan",
      tahun: "$_id.tahun",
      total_ok: 1,
      total_target: 1,
      total_reject: 1,
      availability: { $divide: ["$total_op", "$total_avail"] },
      performance: { $divide: ["$total_ok", "$total_target"] },
      quality: {
        $divide: [
          "$total_ok",
          { $add: ["$total_ok", "$total_reject"] }
        ]
      },
    }
  },
  { $sort: { bulan: 1 } }
])
```

```

OEE: {
$multiply: [
{ $divide: ["$total_ok", "$total_target" ] },
{ $divide: ["$total_op", "$total_avail" ] },
{
$divide: [
"$total_ok",
{ $add: ["$total_ok", "$total_reject" ] }
]
}
]
}
},
{ $sort: { tahun: 1, bulan: 1 } }
])

```

Output

```

{
total_target: 245,
total_ok: 231,
total_reject: 7,
mesin: 'M09',
bulan: 3,
tahun: 2026,
availability: 0.7520833333333333,
performance: 0.9428571428571428,
quality: 0.9705882352941176,
OEE: 0.688251050420168
},
{
total_target: 419,
total_ok: 290,
total_reject: 1,
mesin: 'M02',

```

19

```

bulan: 3,
tahun: 2026,
availability: 0.9520833333333333,
performance: 0.6921241050119332,
quality: 0.9965635738831615,
OEE: 0.6566953582276024
},
{
total_target: 725,
total_ok: 456,
total_reject: 32,

```

```

mesin: 'M05',
bulan: 3,
tahun: 2026,
availability: 0.8052083333333333,
performance: 0.6289655172413793,
quality: 0.9344262295081968,
OEE: 0.4732385528547202
},

```

2. Mesin Terburuk: Tampilkan 3 mesin dengan OEE terendah di bulan Juni 2026 saja (gunakan \$match di awal untuk filter bulan).

```

db.produksi_harian.aggregate([
{
$match: {
tanggal: {
$gte: ISODate("2026-06-01"),
$lt: ISODate("2026-07-01")
}
}
},
{ $group: {
_id: {
mesin: "$mesin",
bulan: { $month: "$tanggal" },
tahun: { $year: "$tanggal" }
},

```

```

20
total_target: { $sum: "$target" },
total_ok: { $sum: "$actual_ok" },
total_reject: { $sum: "$actual_reject" },
total_op: { $sum: "$durasi_operasi_menit" },
total_avail: { $sum: "$durasi_tersedia_menit" }
}
},
{
$project: {
_id: 0,
mesin: "$_id.mesin",
bulan: "$_id.bulan",
tahun: "$_id.tahun",
total_ok: 1,
total_target: 1,
total_reject: 1,
availability: { $divide: ["$total_op", "$total_avail"] },
performance: { $divide: ["$total_ok", "$total_target"] },
quality: {
$divide: [

```

```

"$total_ok",
{ $add: ["$total_ok", "$total_reject"] }
]
},
OEE: {
$multiply: [
{ $divide: ["$total_ok", "$total_target"] },
{ $divide: ["$total_op", "$total_avail"] },
{
$divide: [
"$total_ok",
{ $add: ["$total_ok", "$total_reject"] }
] ]}},
{ $sort: { tahun: 1, bulan: 1 } }
])

```

Output

```

{
total_target: 9060,
total_ok: 6760,
total_reject: 534,
mesin: 'M10',
bulan: 6,
tahun: 2026,
availability: 0.88125,

```

21

```

performance: 0.7461368653421634,
quality: 0.926789141760351,
OEE: 0.6093945490896083
},
{
total_target: 12325,
total_ok: 9671,
total_reject: 815,
mesin: 'M09',
bulan: 6,
tahun: 2026,
availability: 0.8454545454545455,
performance: 0.7846653144016227,
quality: 0.9222773221438108,
OEE: 0.6118377210902525
},
{
total_target: 10680,
total_ok: 8665,
total_reject: 724,
mesin: 'M01',

```

```
bulan: 6,  
tahun: 2026,  
availability: 0.8384027777777778,  
performance: 0.8113295880149812,  
quality: 0.9228884865267867,  
OEE: 0.6277681109990466  
},
```

22

3. Kinerja Harian: Hitung OEE per mesin per hari (gunakan \$dateToString untuk ekstrak tanggal).

```
db.produksi_harian.aggregate([  
  {  
    $addFields : {  
      tanggal_harian : {  
        $dateToString: { format: "%Y-%m-%d", date: "$tanggal" }  
      }  
    }  
  },  
  { $group: {  
    _id: {  
      mesin: "$mesin",  
      tanggal: "$tanggal_harian"  
    },  
    total_target: { $sum: "$target" },  
    total_ok: { $sum: "$actual_ok" },  
    total_reject: { $sum: "$actual_reject" },  
    total_op: { $sum: "$durasi_operasi_menit" },  
    total_avail: { $sum: "$durasi_tersedia_menit" }  
  },  
  {  
    $project: {  
      _id: 0,  
      mesin: "$_id.mesin",  
      tanggal: "$_id.tanggal",  
      availability: { $divide: ["$total_op", "$total_avail"] },  
      performance: { $divide: ["$total_ok", "$total_target"] },  
      quality: {  
        $divide: [  
          "$total_ok",
```

23

```
{ $add: ["$total_ok", "$total_reject"] }  
]  
},
```

```

OEE: {
  $multiply: [
    { $divide: ["$total_ok", "$total_target"] },
    { $divide: ["$total_op", "$total_avail"] },
    {
      $divide: [
        "$total_ok",
        { $add: ["$total_ok", "$total_reject"] }
      ]
    }
  ]
},
{ $sort: { tanggal: 1 } }
])

```

Output

```

{
  mesin: 'M03',
  tanggal: '2026-03-31',
  availability: 0.7125,
  performance: 0.7252252252252253,
  quality: 0.8944444444444445,
  OEE: 0.46217999249249253
},
{
  mesin: 'M08',
  tanggal: '2026-03-31',
  availability: 0.8333333333333334,
  performance: 0.9707903780068728,
  quality: 0.9262295081967213,
  OEE: 0.7493122453195126
},
{

```

24

```

  mesin: 'M05',
  tanggal: '2026-03-31',
  availability: 0.8052083333333333,
  performance: 0.6289655172413793,
  quality: 0.9344262295081968,
  OEE: 0.4732385528547202
},
{

```

4. Simulasi Perbaikan: Jika quality harus minimal 95%, tampilkan mesin yang tidak memenuhi kriteria tersebut.

```

db.produksi_harian.aggregate([
  { $group: {
    _id: "$mesin",
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" }
  }

```

```

},
{$project : {
  "_id" : 0,
  "nama mesin" : "$_id",
  "quality" : {$divide : ["$total_ok", { $add: ["$total_ok",
"$total_reject"] }]}
}},
{
  $match : {
    quality : {$lt : 0.95}
  }
}
])

```

25

Output

```

[
  { 'nama mesin': 'M07', quality: 0.9271430041505162 },
  { 'nama mesin': 'M02', quality: 0.930925893225028 },
  { 'nama mesin': 'M03', quality: 0.9301372304402067 },
  { 'nama mesin': 'M01', quality: 0.9257394011537405 },
  { 'nama mesin': 'M08', quality: 0.9300423131170663 },
  { 'nama mesin': 'M10', quality: 0.9319534199244993 },
  { 'nama mesin': 'M06', quality: 0.9333635539437897 },
  { 'nama mesin': 'M09', quality: 0.9267399267399268 },
  { 'nama mesin': 'M05', quality: 0.9370745360931615 },
  { 'nama mesin': 'M04', quality: 0.930205422667117 }
]

```

PANDUAN TUGAS MANDIRI PERTEMUAN 5

6. Generate Data Dummy (minimal 1000 dokumen untuk inspeksi, 50 untuk target_kualitas)

```
db.inspeksi.drop()
```

```
db.target_kualitas.drop()
```

```
const mesin_list = Array.from({ length: 10 }, (_, i) => `M${String(i+1).padStart(2,'0')}`)
```

```
const batch_ids = Array.from({ length: 2000 }, (_, i) => `B${String(i+1).padStart(4,'0')}`)
```

```
const inspektor_list = ["Andi", "Budi", "Citra", "Dian", "Eka"]
```

```
const jenis_cacat_options = ["gores", "retak", "bengkok", "warna", "dimensi", "pori"]
```

```
// =====
```

```
// Generate data inspeksi
```

```
// =====
```



```

let docs = []
let start_date = new Date(2026, 3, 1, 6, 0) // bulan di JS mulai dari 0 (3 = April)

for (let i = 0; i < 1500; i++) {
  let batch = batch_ids[Math.floor(Math.random() * batch_ids.length)]
  let mesin = mesin_list[Math.floor(Math.random() * mesin_list.length)]

  let tanggal = new Date(start_date)
  tanggal.setMinutes(tanggal.getMinutes() + Math.floor(Math.random() * 60 * 24 * 90))

  let jam = tanggal.getHours()
  let shift = jam < 14 ? 1 : (jam < 22 ? 2 : 3)

  let jumlah = Math.floor(Math.random() * 401) + 100
  let reject = Math.floor(jumlah * (Math.random() * 0.10))

  let shuffled = [...jenis_cacat_options].sort(() => 0.5 - Math.random())
  let jenis = shuffled.slice(0, Math.floor(Math.random() * 3) + 1)

  docs.push({
    batch_id: batch,
    mesin: mesin,
    tanggal: tanggal,
    shift: shift,
    inspektor: inspektor_list[Math.floor(Math.random() * inspektor_list.length)],
    jumlah_diperiksa: jumlah,
    cacat_ditemukan: reject,
    jenis_cacat: jenis
  })

  if (docs.length >= 500) {
    db.inspeksi.insertMany(docs)
    docs = []
  }
}

if (docs.length > 0) {

```

```

db.inspeksi.insertMany(docs)
}

print("Data inspeksi selesai. Total:", db.inspeksi.countDocuments())

// =====
// Generate target kualitas
// =====

let targets = []

for (let i = 1; i <= 2000; i++) {
  let batch = `B${String(i).padStart(4,'0')}`
  let target_cacat = Math.floor(Math.random() * 15) + 1

  targets.push({
    batch_id: batch,
    target_maks_cacat: target_cacat
  })

  if (targets.length >= 500) {
    db.target_kualitas.insertMany(targets)
    targets = []
  }

  if (targets.length > 0) {
    db.target_kualitas.insertMany(targets)
  }

  print("Data target kualitas selesai. Total:", db.target_kualitas.countDocuments())
}

```

```

< Data inspeksi selesai. Total:
< 1500
< Data target kualitas selesai. Total:
< 2000

```

7. Memastikan data siap

```
db.inspeksi.countDocuments()
```

```
1500
```

```
db.target_kualitas.countDocuments()
```

```
2000
```

```
db.inspeksi.findOne()
```

```
< {
  _id: ObjectId('69f8370293634fca2b1b8abf'),
  batch_id: 'B0148',
  mesin: 'M05',
  tanggal: 2026-06-23T13:57:00.000Z,
  shift: 2,
  inspektor: 'Eka',
  jumlah_diperiksa: 165,
  cacat_ditemukan: 13,
  jenis_cacat: [
    'gores'
  ]
}
```

```
db.target_kualitas.findOne()
```

```
< {
  _id: ObjectId('69f8370293634fca2b1b909b'),
  batch_id: 'B0001',
  target_maks_cacat: 1
}
```

```
db.inspeksi.createIndex({ batch_id: 1 })
```

```
db.inspeksi.createIndex({ mesin: 1, tanggal: 1 })
```

```
db.target_kualitas.createIndex({ batch_id: 1 })
```

```

> db.inspeksi.createIndex({ batch_id: 1 })
< batch_id_1
> db.inspeksi.createIndex({ mesin: 1, tanggal: 1 })
< mesin_1_tanggal_1
> db.target_kualitas.createIndex({ batch_id: 1 })
< batch_id_1

```

Untuk selanjutnya ada pipeline 1-5 yang dapat dilihat pada bagian Pipeline dan Hasil

Latihan Mandiri

1. Gunakan database latihan5, koleksi data_sensor dengan 5000 dokumen (field: sensor_id, nilai, timestamp, category).

```
db.data_sensor.drop()
```

```
db.sensor_info.drop()
```

```
const categories = ["suhu", "tekanan", "getaran"]
```

```
let docs = []
```

```
let start = new Date()
```

```
for (let i = 0; i < 5000; i++) {
```

```
  docs.push({
```

```
    sensor_id: "S" + String(Math.floor(Math.random()*20)+1).padStart(2,'0'),
```

```
    nilai: Math.floor(Math.random() * 101), // 0–100
```

```
    timestamp: new Date(start.getTime() - Math.random()*7*24*60*60*1000),
```

```
    category: categories[Math.floor(Math.random()*categories.length)]
```

```
  })
```

```
if (docs.length >= 500) {
```

```
  db.data_sensor.insertMany(docs)
```

```
  docs = []
```

```
}
```

```
}
```

```
if (docs.length) db.data_sensor.insertMany(docs)
```

```
print("Total data_sensor:", db.data_sensor.countDocuments())
```

```
< Total data_sensor:
< 5000
```

2. Tulis pipeline untuk:

- a. Menampilkan rata-rata nilai per sensor_id pada 7 hari terakhir.
- b. Menampilkan sensor yang memiliki nilai di atas 90 minimal 3 kali (gunakan \$group dengan \$push atau \$sum kondisional).
- c. Menambahkan field jam dan menghitung nilai rata-rata per jam per sensor.

a. Pipeline 1

```
db.data_sensor.aggregate([
  {
    $match: {
      timestamp: {
        $gte: new Date(Date.now() - 7*24*60*60*1000)
      }
    }
  },
  {
    $group: {
      _id: "$sensor_id",
      rata_nilai: { $avg: "$nilai" }
    }
  },
  {
    $project: {
      _id: 0,
```

```
    sensor_id: "$_id",  
    rata_nilai: 1  
  }  
}  
D)
```

```
{
  rata_nilai: 52.43852459016394,
  sensor_id: 'S08'
}
{
  rata_nilai: 52.559485530546624,
  sensor_id: 'S02'
}
{
  rata_nilai: 51.224806201550386,
  sensor_id: 'S14'
}
{
  rata_nilai: 48.6597510373444,
  sensor_id: 'S15'
}
{
  rata_nilai: 54.133640552995395,
  sensor_id: 'S04'
}
{
  rata_nilai: 50.48627450980392,
  sensor_id: 'S19'
}
{
  rata_nilai: 50.44979919678715,
  sensor_id: 'S07'
}
{
  rata_nilai: 49.8996138996139,
  sensor_id: 'S03'
}
```

b. Pipeline 2

```
db.data_sensor.aggregate([
  {
    $group: {
      _id: "$sensor_id",
      jumlah_lebih_90: {
        $sum: {
          $cond: [{ $gt: ["$nilai", 90] }, 1, 0]
        }
      }
    }
  },
  {
    $match: {
      jumlah_lebih_90: { $gte: 3 }
    }
  },
  {
    $project: {
      _id: 0,
      sensor_id: "$_id",
      jumlah_lebih_90: 1
    }
  }
])
```



```
< {  
  jumlah_lebih_90: 24,  
  sensor_id: 'S14'  
}  
{  
  jumlah_lebih_90: 34,  
  sensor_id: 'S18'  
}  
{  
  jumlah_lebih_90: 23,  
  sensor_id: 'S08'  
}  
{  
  jumlah_lebih_90: 35,  
  sensor_id: 'S02'  
}  
{  
  jumlah_lebih_90: 20,  
  sensor_id: 'S04'  
}  
{  
  jumlah_lebih_90: 27,  
  sensor_id: 'S19'  
}  
{  
  jumlah_lebih_90: 24,  
  sensor_id: 'S03'  
}  
{  
  jumlah_lebih_90: 30,  
  sensor_id: 'S07'  
}  
{  
  jumlah_lebih_90: 19,  
  sensor_id: 'S20'
```

c. Pipeline 3

```
db.data_sensor.aggregate([
  {
    $addFields: {
      jam: { $hour: "$timestamp" }
    }
  },
  {
    $group: {
      _id: {
        sensor_id: "$sensor_id",
        jam: "$jam"
      },
      rata_nilai: { $avg: "$nilai" }
    }
  },
  {
    $project: {
      _id: 0,
      sensor_id: "$_id.sensor_id",
      jam: "$_id.jam",
      rata_nilai: 1
    }
  },
  {
    $sort: { sensor_id: 1, jam: 1 }
  }
])
```

```
< {
  rata_nilai: 42.1,
  sensor_id: 'S01',
  jam: 0
}
{
  rata_nilai: 56,
  sensor_id: 'S01',
  jam: 1
}
{
  rata_nilai: 50.833333333333336,
  sensor_id: 'S01',
  jam: 2
}
{
  rata_nilai: 52.38461538461539,
  sensor_id: 'S01',
  jam: 3
}
{
  rata_nilai: 52.25,
  sensor_id: 'S01',
  jam: 4
}
{
  rata_nilai: 56.888888888888886,
  sensor_id: 'S01',
  jam: 5
}
{
```

3. Buat koleksi sensor_info lalu join untuk menampilkan nama sensor dalam hasil.

```
db.sensor_info.insertMany([
  { sensor_id: "S01", nama: "Sensor Suhu A" },
  { sensor_id: "S02", nama: "Sensor Suhu B" },
  { sensor_id: "S03", nama: "Sensor Tekanan A" },
  { sensor_id: "S04", nama: "Sensor Tekanan B" },
  { sensor_id: "S05", nama: "Sensor Getaran A" }
])
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f8550f93634fca2b1bbf7b'),
    '1': ObjectId('69f8550f93634fca2b1bbf7c'),
    '2': ObjectId('69f8550f93634fca2b1bbf7d'),
    '3': ObjectId('69f8550f93634fca2b1bbf7e'),
    '4': ObjectId('69f8550f93634fca2b1bbf7f')
  }
}
```

```
db.data_sensor.aggregate([
  {
    $group: {
      _id: "$sensor_id",
      rata_nilai: { $avg: "$nilai" }
    }
  },
  {
    $lookup: {
      from: "sensor_info",
      localField: "_id",
      foreignField: "sensor_id",
      as: "info"
    }
  },
  {
    $unwind: {
```

```
    path: "$info",
    preserveNullAndEmptyArrays: true
  }
},
{
  $project: {
    _id: 0,
    sensor_id: "$_id",
    nama_sensor: "$info.nama",
    rata_nilai: 1
  }
}
])
```

>_MONGOSH

```
< {
  rata_nilai: 47.96590909090909,
  sensor_id: 'S05',
  nama_sensor: 'Sensor Getaran A'
}
{
  rata_nilai: 49.9601593625498,
  sensor_id: 'S11'
}
{
  rata_nilai: 49.05860805860806,
  sensor_id: 'S15'
}
{
  rata_nilai: 47.57707509881423,
  sensor_id: 'S14'
}
{
  rata_nilai: 51.752941176470586,
  sensor_id: 'S19'
}
{
  rata_nilai: 51.99186991869919,
  sensor_id: 'S10'
}
{
  rata_nilai: 47.41532258064516,
  sensor_id: 'S18'
}
{
  rata_nilai: 50.13304721030043,
  sensor_id: 'S02',
  nama_sensor: 'Sensor Suhu B'
}
{
```

4. Dokumentasikan setiap pipeline, jelaskan setiap stage.
 - Pipeline 1: Rata-rata Nilai per Sensor (7 Hari Terakhir)

Query:

```
db.data_sensor.aggregate([
  {
    $match: {
      timestamp: {
        $gte: new Date(Date.now() - 7*24*60*60*1000)
      }
    }
  },
  {
    $group: {
      _id: "$sensor_id",
      rata_nilai: { $avg: "$nilai" }
    }
  },
  {
    $project: {
      _id: 0,
      sensor_id: "$_id",
      rata_nilai: 1
    }
  }
])
```

Penjelasan:

\$match

Menyaring data agar hanya mengambil data dalam 7 hari terakhir berdasarkan field timestamp.

\$group

Mengelompokkan data berdasarkan sensor_id, kemudian menghitung rata-rata nilai (\$avg) untuk setiap sensor.

\$project

Mengatur format output:

Menghilangkan _id

Mengganti _id menjadi sensor_id

Menampilkan field rata_nilai

- Pipeline 2: Sensor dengan Nilai > 90 Minimal 3 Kali

Query:

```
db.data_sensor.aggregate([
  {
    $group: {
      _id: "$sensor_id",
      jumlah_lebih_90: {
        $sum: {
          $cond: [{ $gt: ["$nilai", 90] }, 1, 0]
        }
      }
    },
    {
      $match: {
        jumlah_lebih_90: { $gte: 3 }
      }
    },
    {
      $project: {
        _id: 0,
        sensor_id: "$_id",
        jumlah_lebih_90: 1
      }
    }
  ])
```

Penjelasan:

\$group

Mengelompokkan data berdasarkan sensor_id.

\$cond

Mengecek kondisi:

Jika nilai > 90 → dihitung sebagai 1

Jika tidak → 0

\$sum

Menjumlahkan hasil kondisi untuk mendapatkan berapa kali nilai > 90 terjadi.

\$match

Memfilter sensor yang memiliki kejadian nilai > 90 minimal 3 kali.

\$project

Menampilkan hasil akhir dengan format yang rapi.

- Pipeline 3: Rata-rata Nilai per Jam per Sensor

Query:

```
db.data_sensor.aggregate([
  {
    $addFields: {
      jam: { $hour: "$timestamp" }
    }
  },
  {
    $group: {
      _id: {
        sensor_id: "$sensor_id",
        jam: "$jam"
      },
      rata_nilai: { $avg: "$nilai" }
    }
  },
  {
    $project: {
      _id: 0,
      sensor_id: "$_id.sensor_id",
      jam: "$_id.jam",
      rata_nilai: 1
    }
  },
  {
    $sort: { sensor_id: 1, jam: 1 }
  }
])
```

Penjelasan:

\$addFields

Menambahkan field baru jam yang diambil dari timestamp menggunakan \$hour.

\$group

Mengelompokkan data berdasarkan kombinasi:

sensor_id

jam

lalu menghitung rata-rata nilai.

\$project

Menyusun ulang output agar mudah dibaca.

\$sort

Mengurutkan hasil berdasarkan sensor_id dan jam.

- Pipeline 4: Join dengan Koleksi sensor_info

Query:

```
db.data_sensor.aggregate([
  {
    $group: {
      _id: "$sensor_id",
      rata_nilai: { $avg: "$nilai" }
    }
  },
  {
    $lookup: {
      from: "sensor_info",
      localField: "_id",
      foreignField: "sensor_id",
      as: "info"
    }
  },
  {
    $unwind: {
      path: "$info",
      preserveNullAndEmptyArrays: true
    }
  },
  {
    $project: {
      _id: 0,
      sensor_id: "$_id",
      nama_sensor: "$info.nama",
```

```
        rata_nilai: 1
    }
}
])
```

Penjelasan:

\$group

Menghitung rata-rata nilai per sensor_id.

\$lookup

Menggabungkan data dari koleksi sensor_info berdasarkan:

sensor_id pada kedua koleksi

\$unwind

Mengubah hasil array dari \$lookup menjadi objek tunggal.

preserveNullAndEmptyArrays: true

Agar data tetap muncul walaupun tidak memiliki pasangan di sensor_info.

\$project

Menampilkan:

sensor_id

nama_sensor

rata_nilai